Foundation of Embodied AI: Course Map

2026-10-30

Contents

Six learning modules 2
Embodied AI system map 3
Reading paths 3
Tutorial overview 4

简体中文 | **English** | PDF

Item	Details
Topic	Course Map
Revision	2026-10-30
License	CC BY-NC-SA 4.0 ·Xiong Zeng and contributors

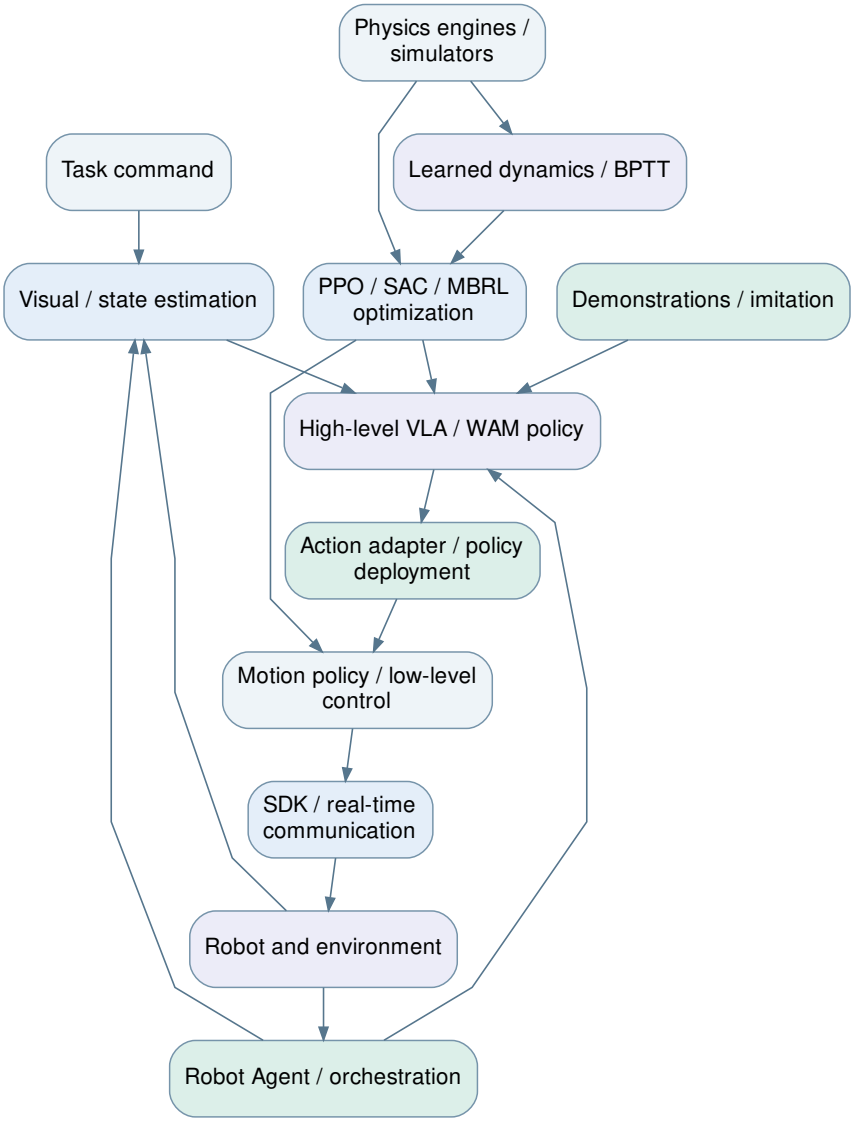
Six learning modules

Twenty topics are organized as tools → calibration and deep learning → physics, perception and control → robot RL → demonstrations and generative policies → communication and deployment → systems integration. Long-form derivations remain optional deep dives rather than strict prerequisites.

Tutorial numbers 1–20 match the course order in the README.

Module	Topics	Outcome
0. Orientation	Systems map and guide	Explain perception–policy–control loops
1. Tools	Git, ROS 2	Manage code and understand ROS communication and TF
2. Physics, perception and neural foundations	Calibration, Deep Learning Basics, vision foundation models, dynamics, low-level control, Simulation & RL	Relate calibrated geometry, neural networks, physical models, actuator control and simulation
3. Reinforcement learning	Motion RL, PPO, SAC, Model-Based RL	Design observations, actions, rewards and data collection
4. Demonstrations and models	Imitation Learning, Flow Matching, GR00T / SONIC, World Action Models, Efficient Training & Inference, Post-training	Learn generative and predictive policies and connect efficient inference with hierarchical controllers
5. Deployment and integration	Communication & Real-Time Systems, Robot Agents	Connect synchronized sensing, inference and actuator control, then orchestrate long-horizon tasks

Embodied AI system map



Reading paths

1. Beginners: orientation → essential Git and ROS 2 → calibration → deep learning → dynamics → low-level control → vision foundation models → simulation.
2. Motion control: dynamics → low-level control → simulation → deep learning → PPO / SAC → motion control RL → Model-Based RL → communication and real-time systems.
3. VLA: calibration → deep learning → vision foundation models → Imitation Learning → Flow Matching → GR00T / SONIC → World Action Models → efficient training and inference → post-training → communication and real-time systems → Robot Agents.
4. Deployment prerequisites: complete ROS 2 and low-level control before communication and real-time systems; combine a trained motion or VLA policy with these foundations for systems integration. Robot Agents then coordinate

the full system.

5. Mathematical deep dives: full PPO, SAC, and Flow Matching derivations are reference material; hands-on learning does not require deriving everything first.

Tutorial overview

1. Git Basics

Git Basics — Learn Git's working tree, staging area and commit history, then follow a GitHub workflow covering branches, pull requests, conflict resolution and common recovery commands.

2. ROS 2 Basics

ROS 2 Basics — Understand nodes, topics, services, actions, QoS and TF. Connect sensors, state estimation, policy inference and controllers through ROS 2 interfaces.

3. Camera Calibration

Camera Calibration — Study camera intrinsics, distortion, extrinsics and hand–eye calibration. Relate RGB-D observations and coordinate transforms to robot-frame object poses.

4. Deep Learning Basics

Deep Learning Basics — Follow a robot task from linear models and MLPs through losses, backpropagation and reliable training. Study CNN/ViT visual tokens, Q/K/V and hand-worked Attention, Transformer blocks and multimodal action heads; connect pretraining, SFT/LoRA and RL post-training to deployment timing and controller interfaces. BPTT and differentiable world models form an advanced appendix.

5. Vision Foundation Models

Vision Foundation Models — Compare segmentation, pose estimation, stereo depth and 3D foundation models. Follow four robotics workflows spanning known objects, stereo geometry, unknown shapes and human demonstrations.

6. Robot Dynamics

Robot Dynamics — Build physical intuition from a grasp: actuator response, mass and inertia, joint coupling, Jacobian force mapping, contact and friction. Connect fixed- and floating-base dynamics to low-level control, simulator parameters and diagnosis of sim-to-real failures.

7. Low-Level Control

Low-Level Control — Connect motor servo loops, PD/PID and MIT Mode to impedance, admittance and contact control. Relate policy action semantics to actuator response, simulated hardware alignment and protected tuning.

8. Simulation & RL

Simulation & RL — Distinguish model descriptions, physics solvers, scenes and RL task frameworks across Isaac Sim/Lab, MuJoCo/mjlab, Genesis World and Newton. Follow robot modeling, observations/actions/rewards/resets, training and evaluation through sim-to-real interface alignment; compare tools by task requirements rather than a universal speed ranking.

9. Motion Control RL

Motion Control RL — Define closed-loop motion tasks through observations, goals, actions, rewards and resets. Distinguish from-scratch RL, Motion Tracking and Residual RL from PPO/SAC and model-based optimization; connect reference motions and generalist controllers to teacher–student learning, RMA and Sim-to-Real. Use G1 velocity tracking and TRON2/Revo3 screwdriver manipulation to organize evaluation of success, generalization and recovery.

10. PPO / Actor–Critic

PPO / Actor–Critic — Derive PPO-Clip and Actor–Critic learning from policy gradients, Bellman targets and GAE. Explain Gaussian policy updates, gradient isolation and asymmetric robot observations.

11. Q-learning / SAC

Q-learning / SAC — Trace Bellman equations, TD/Q-learning and fitted Q-iteration toward DQN and continuous-action actor–critic methods. Derive modern SAC's critic, actor and temperature losses; compare PPO, DQN and SAC; connect HIL-SERL and VLA/WAM action-chunk value learning, with mathematical details in appendices.

12. Model-Based RL

Model-Based RL — Study Dyna, PETS, Dreamer, SHAC and SuperMimic. Derive BPTT through learned dynamics, connect truncated actor–critic updates and model bias, and distinguish data augmentation, planning and differentiable policy learning.

13. Imitation Learning

Imitation Learning — Study BC, DAgger and corrective demonstrations, then connect ACT, Diffusion Policy and Flow Matching to action chunks. Explain VLA architectures and supervised action learning, temporal ensembling, and inference-time versus training-time RTC.

14. Flow Matching

Flow Matching — Derive conditional Flow Matching, velocity fields and the training objective. Explain how ODE integration transforms noise into conditional robot action sequences.

15. GR00T & SONIC

GR00T & SONIC — Use GR00T and SONIC to explain System 1/System 0 humanoid control. Connect high-level action chunks, motion representations, whole-body policies and low-level execution.

16. World Action Models

World Action Models — Distinguish world models, VLAs and WAMs, then study DreamZero training and accelerated inference. Explain Dream-1 zero-shot embodiment transfer through shared EEF/IK interfaces, joint video–action modeling and transferable visual representations; compare FaaS adaptation, VLA/WAM compute, latency, contact control and evaluation. Distinguishes SuperMimic state-dynamics models from visual WAMs.

17. Efficient Training & Inference

Efficient Training & Inference — Connect Transformer and VLA/WAM training costs, FlashAttention and valid caching to DreamZero acceleration, few-step inference, asynchronous robot control, RTC and cloud latency.

18. Post-training

Post-training — Compare SFT and RL for pretrained VLA and WAM models. Study six VLA RL routes, WAM-RL with online video SFT and Flow-SDE, then connect actor–critic semantics, RLinf integration and robot deployment limits.

19. Communication & Real-Time Systems

Communication & Real-Time Systems — Separate ROS 2/DDS, SDKs and CAN/EtherCAT buses. Explain timestamps, data freshness, latency and jitter, multirate scheduling, asynchronous inference and fault handling in robot feedback loops.

20. Robot Agents

Robot Agents — Distinguish foundation models, coding agents and agent harnesses. Connect RAX exploration, ASPIRE trace repair and RPent toolkits, memory, Flash Mode and the RLinf data flywheel to reusable robot workflows.